Python Dictionaries & Sets

key : value · unique items · union · intersection

Examples · Predict the output · Step-by-step coding · Tasks

```
{"key": value}
```

```
d.get("age")
```

```
{1, 2, 3}
```

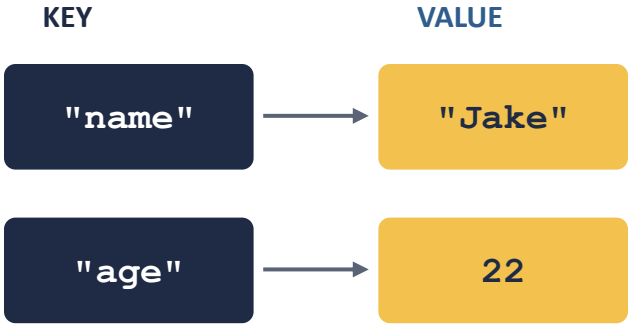
```
a | b   a & b
```

What is a dictionary?

```
data = {"name": "Jake", "age": 22}
print(data)
print(type(data))
print(len(data))
```

Output

```
{'name': 'Jake', 'age': 22}
<class 'dict'>
2
```



- Look up a value by its KEY
- Keys are unique
- Keys must be immutable
- Values can be anything

Creating a dictionary

```
a = {"x": 1, "y": 2}
b = dict(name="Sam", age=20)
c = dict([("p", 5), ("q", 6)])
e = {}
print(a)
print(b)
print(c)
print(e, type(e))
```

Output

```
{'x': 1, 'y': 2}
{'name': 'Sam', 'age': 20}
{'p': 5, 'q': 6}
{} <class 'dict'>
```

- {key: value, ...}
- dict(key=value, ...)
- dict(list of pairs)
- {} is an EMPTY DICT, not a set
- Keys: str, int, tuple (immutable)
- Values: anything

Accessing items: [ ] and get()

```
d = {"name": "Kat", "age": 21}
print(d["name"])
print(d.get("age"))
print(d.get("city"))
print(d.get("city", "NA")) # missing -> default
print("age" in d)
```

Output

```
Kat
21
None
NA
True
```

Syntax	Meaning
`d[key]`	value; KeyError if missing
`d.get(key)`	value; None if missing
`d.get(key, x)`	value; x if missing
`key in d`	True / False

❌ **Error example**

```
d = {"name": "Kat"}
print(d["city"])
KeyError: 'city'
```

Adding and updating items

```
d = {"name": "Sam"}
d["age"] = 21
print(d)
d["name"] = "Alex"
print(d)
d.update({"city": "Kochi", "age": 22})
print(d)
```

Output

```
{'name': 'Sam', 'age': 21}
{'name': 'Alex', 'age': 21}
{'name': 'Alex', 'age': 22, 'city': 'Kochi'}
```

start



d['age'] = 21 → new key, ADDED



d['name'] = 'Alex' → key exists, UPDATED



Same syntax: new key adds, existing key updates

Removing items

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}
del d["a"]              # delete by key
print(d)
val = d.pop("b")
print(val, d)
print(d.popitem())
print(d)
d.clear()
print(d)
```

Output

```
{'b': 2, 'c': 3, 'd': 4}
2 {'c': 3, 'd': 4}
('d', 4)
{'c': 3}
{}
```

`del d['a']`	~~a~~	b	c	d	*just deletes*
`d.pop('b')`	~~b~~	c	d		*returns 2*
`d.popitem()`	~~c~~	~~d~~			*returns ('d', 4)*
`d.clear()`	~~c~~				*→ {}*

pink = removed in that step

🤔 Predict the output

```
d = {"a": 1, "b": 2}
d["a"] = 10
d["c"] = 3
print(d)
print(len(d))
print(d.pop("b") + d["c"])
```

?

Write your answer first. Then we run it!

✅ Let's run it

```
d = {"a": 1, "b": 2}
d["a"] = 10
d["c"] = 3
print(d)
print(len(d))
print(d.pop("b") + d["c"])
```

Output

```
{'a': 10, 'b': 2, 'c': 3}
3
5
```

- 'a' exists, so it is updated to 10
- 'c' is new, so it is added at the end
- 3 pairs remain
- pop returns 2; 2 + 3 = 5

Looping through a dictionary

```
d = {"a": 1, "b": 2, "c": 3}
for key in d:                # keys
    print(key, end=" ")
print()
for value in d.values():     # values
    print(value, end=" ")
print()
for key, value in d.items():    # pairs
    print(key, "->", value)
```

Output

```
a b c
1 2 3
a -> 1
b -> 2
c -> 3
```

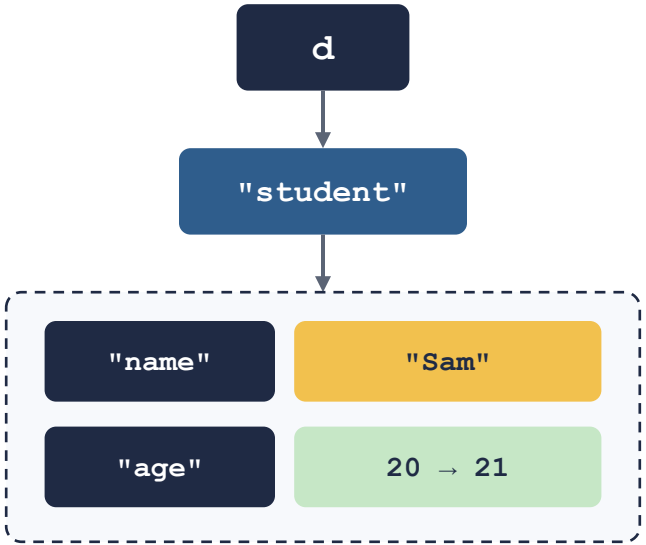
Syntax	Meaning
`for k in d`	loop over keys
`d.keys()`	all keys
`d.values()`	all values
`d.items()`	all (key, value) pairs

Nested dictionaries

```
d = {
    "student": {
        "name": "Sam",
        "age": 20
    }
}
print(d["student"])
print(d["student"]["name"])
d["student"]["age"] = 21
print(d)
```

Output

```
{'name': 'Sam', 'age': 20}
Sam
{'student': {'name': 'Sam', 'age': 21}}
```



`d["student"]["name"]` → `"Sam"`

Part B: Sets

unique · unordered · frozenset · add/remove · union · intersection · difference

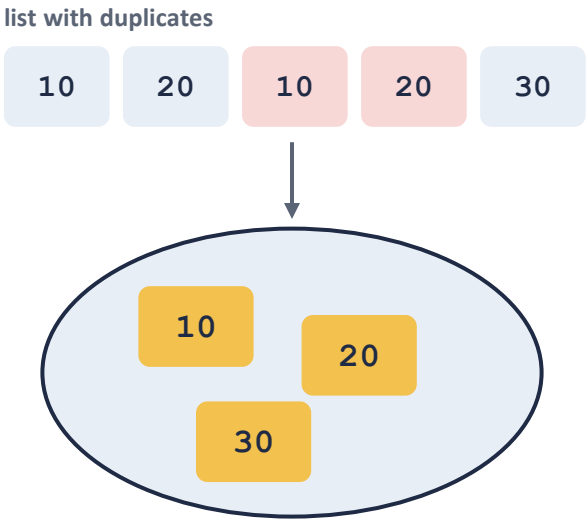
⏱ 30 min

What is a set?

```
s = {10, 50, 20}
print(s)
print(type(s))
s2 = {10, 20, 10, 20, 30}
print(s2)
```

Output

```
{10, 50, 20}
<class 'set'>
{10, 20, 30}
```



set: unique, no order, no index

Creating sets (type casting)

```
a = set([1, 2, 2, 3])
b = set((4, 5, 4))
c = set("hello")
e = set()                    # EMPTY set
print(a, b)
print(sorted(c))
print(e, type({}))
```

Output

```
{1, 2, 3} {4, 5}
['e', 'h', 'l', 'o']
set() <class 'dict'>
```

- set(list), set(tuple), set(string)
- Duplicates disappear
- Empty set: set()
- {} is a dictionary, not a set!
- Order may differ on your computer

Frozen sets

```
s = set(["a", "b", "c"])
fs = frozenset(["e", "f", "g"])
print(type(s).__name__, type(fs).__name__)
print("e" in fs, len(fs))
s.add("d")
print(sorted(s))
fs.add("h")
```

Output

```
set frozenset
True 3
['a', 'b', 'c', 'd']
AttributeError: 'frozenset' object has no attribute 'add'
```

- frozenset = an immutable set
- Can be read and tested with 'in'
- No add / remove
- Can be used as a dictionary key

Adding and removing items

```
s = {1, 2, 3}
s.add(4)              # one item
s.update([5, 6])      # many items
print(s)
s.remove(2)           # error if missing
s.discard(99)
print(s)
s.clear()
print(s)
```

Output

```
{1, 2, 3, 4, 5, 6}
{1, 3, 4, 5, 6}
set()
```

Syntax	Meaning
`add(x)`	add one item
`update([..])`	add many items
`remove(x)`	delete; KeyError if missing
`discard(x)`	delete; no error if missing
`clear()`	empty the set

❌ **Error example**

```
s = {1, 2}
s.remove(5)
KeyError: 5
```

🤔 Predict the output

```
s = {1, 2, 2, 3}
s.add(2)
s.add(4)
print(len(s))
s.discard(10)
print(s)
print(sum(s), max(s))
```

?

Write your answer first. Then we run it!

✅ Let's run it

```
s = {1, 2, 2, 3}
s.add(2)
s.add(4)
print(len(s))
s.discard(10)
print(s)
print(sum(s), max(s))
```

Output

```
4
{1, 2, 3, 4}
10 4
```

- {1, 2, 2, 3} is {1, 2, 3}
- adding 2 again changes nothing; adding 4 → 4 items
- discard(10) does nothing, with no error
- sum = 10, max = 4

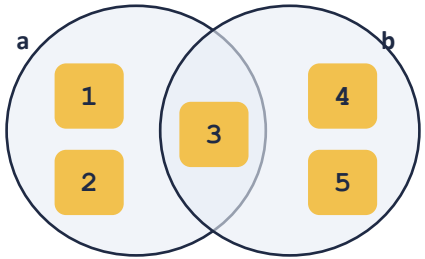
Set operations: union, intersection, difference

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.union(b), a | b)
print(a.intersection(b), a & b)
print(a.difference(b), a - b)
print(b - a)
print(a.symmetric_difference(b), a ^ b)
```

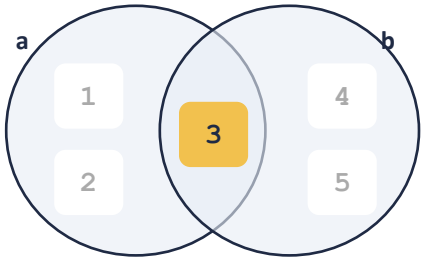
Output

```
{1, 2, 3, 4, 5} {1, 2, 3, 4, 5}
{3} {3}
{1, 2} {1, 2}
{4, 5}
{1, 2, 4, 5} {1, 2, 4, 5}
```

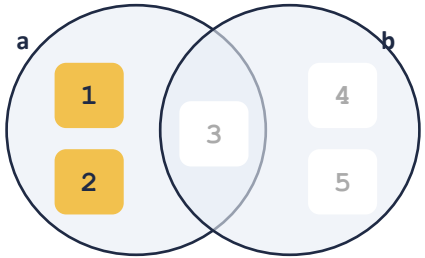
`a | b` union



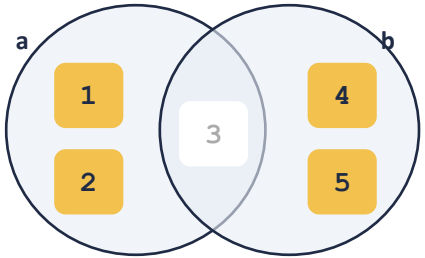
`a & b` intersection



`a - b` difference



`a ^ b` symmetric diff.



Comparing sets

```
a = {1, 2}
b = {1, 2, 3}
print(a <= b, a.issubset(b))
print(b >= a, b.issuperset(a))
print({1, 2} == {2, 1})
print(a.isdisjoint({7, 8}))
print(3 in b, 3 not in a)
```

Output

```
True True
True True
True
True
True True
```

Syntax	Meaning
`x in s`	membership
`a <= b`	a is a subset of b
`a >= b`	a is a superset of b
`a == b`	same items
`a.isdisjoint(b)`	no common items

🤔 Predict the output

```
a = {1, 2, 3}
b = {3, 4}
print(a | b)
print(a & b, a - b, b - a)
print(a ^ b)
print({1, 2} <= a)
```

?

Write your answer first. Then we run it!

✅ Let's run it

```
a = {1, 2, 3}
b = {3, 4}
print(a | b)
print(a & b, a - b, b - a)
print(a ^ b)
print({1, 2} <= a)
```

Output

```
{1, 2, 3, 4}
{3} {1, 2} {4}
{1, 2, 4}
True
```

- union: every item once → {1, 2, 3, 4}
- only 3 is common; a − b = {1, 2}; b − a = {4}
- symmetric difference: in one but not both → {1, 2, 4}
- {1, 2} is inside a → True

🧱 Step by step: Remove duplicates from marks

1. A list with repeated marks

```
marks = [78, 85, 78, 92, 85, 60]
```

2. Convert to a set to drop duplicates

```
unique = set(marks)
```

3. Sort them for a neat display

```
ordered = sorted(unique)
```

4. Count how many were removed

```
removed = len(marks) - len(unique)
```

5. Print the results

```
print("Unique marks:", ordered)
print("Duplicates removed:", removed)
```

6. Highest and lowest

```
print("Highest:", max(unique), "Lowest:",
min(unique))
```

🤔 Predict the output

```
print(sorted(set("banana")))
print(len(set()))
print({1, 2} == {2, 1, 1})
s = frozenset([3, 1, 2])
print(sorted(s), 2 in s)
```

?

Write your answer first. Then we run it!

✅ Let's run it

```
print(sorted(set("banana")))
print(len(set()))
print({1, 2} == {2, 1, 1})
s = frozenset([3, 1, 2])
print(sorted(s), 2 in s)
```

Output

```
['a', 'b', 'n']
0
True
[1, 2, 3] True
```

- unique letters of 'banana': a, b, n
- an empty set has length 0
- order and duplicates don't matter, so they are equal
- a frozenset can be sorted and searched

THANKYOU!